PROJET DE FIN D'ANNÉE · RAPPORT TECHNIQUE

Mise en place d'une plateforme d'orchestration et d'automatisation de la réponse aux incidents de sécurité

Détection des intrusions, blocage automatique de l'attaquant, gestion des incidents et threat intelligence avec des outils open source.

pfSense · Snort · Shuffle · TheHive · Cortex · MISP · PostgreSQL · Grafana · Docker · VMware

Réalisé par MAAZOUN Nour · CHOURA Yessine · TOUMI Karim · KALLEL Halim

Année 2025 / 2026

Sommaire

1. Introduction
2. Architecture de la plateforme
3. Détection : pfSense, DMZ et Snort
4. Réponse automatisée avec Shuffle
5. Gestion des incidents avec TheHive
6. Analyse des observables avec Cortex
7. Threat Intelligence avec MISP
8. Scénario de validation de bout en bout
9. Difficultés rencontrées et solutions
10. Conclusion et perspectives

1. Introduction

1.1 Contexte

Un centre opérationnel de sécurité (SOC) reçoit chaque jour un grand nombre d'alertes. Les traiter manuellement prend du temps : entre la détection d'une attaque et la réaction d'un analyste, l'attaquant peut poursuivre son action. Les plateformes SOAR (*Security Orchestration, Automation and Response*) répondent à ce problème en automatisant les premières actions de réponse, tandis que les plateformes SIRP (*Security Incident Response Platform*) organisent le travail des analystes.

Ce projet met en place, dans un laboratoire virtualisé, une plateforme complète qui détecte une attaque, bloque l'attaquant automatiquement, crée l'incident pour les analystes et l'enrichit grâce à la threat intelligence.

1.2 Objectifs

Objectif	Description
SOAR avec Shuffle	Orchestrer la réponse aux attaques sans intervention humaine immédiate : blocage automatique de l'attaquant sur le pare-feu.
SIRP avec TheHive	Structurer la gestion des incidents : alertes, cas, tâches et collaboration entre analystes.
Threat Intelligence avec MISP et Cortex	Qualifier la dangerosité des menaces par l'enrichissement des observables et le partage des indicateurs de compromission (IOC).

1.3 Résultat obtenu

La chaîne complète est opérationnelle :

1. un attaquant lance une attaque contre le serveur exposé dans la DMZ ;
2. l'IDS Snort détecte l'attaque et transmet l'alerte à Shuffle ;
3. Shuffle bloque l'attaquant sur pfSense en moins d'une minute, journalise l'incident dans Grafana, envoie un e-mail et crée une alerte dans TheHive ;
4. l'analyste traite l'incident dans TheHive, l'enrichit avec Cortex et enregistre l'attaquant confirmé dans MISP ;
5. lors des analyses suivantes, Cortex reconnaît automatiquement cet attaquant grâce à MISP.

1.4 Environnement

Le laboratoire est hébergé sur un poste MSI Thin GF63 (Intel i5-12450H, 8 cœurs, 32 Go de RAM) sous VMware Workstation. Tous les composants sont open source ; TheHive, Cortex et MISP sont déployés avec Docker.

2. Architecture de la plateforme

Le laboratoire est découpé en trois zones séparées par le pare-feu pfSense : le **WAN**, où se trouve l'attaquant ; la **DMZ**, qui héberge le serveur cible et l'IDS ; la zone **SOC**, qui regroupe les outils de réponse et d'analyse.

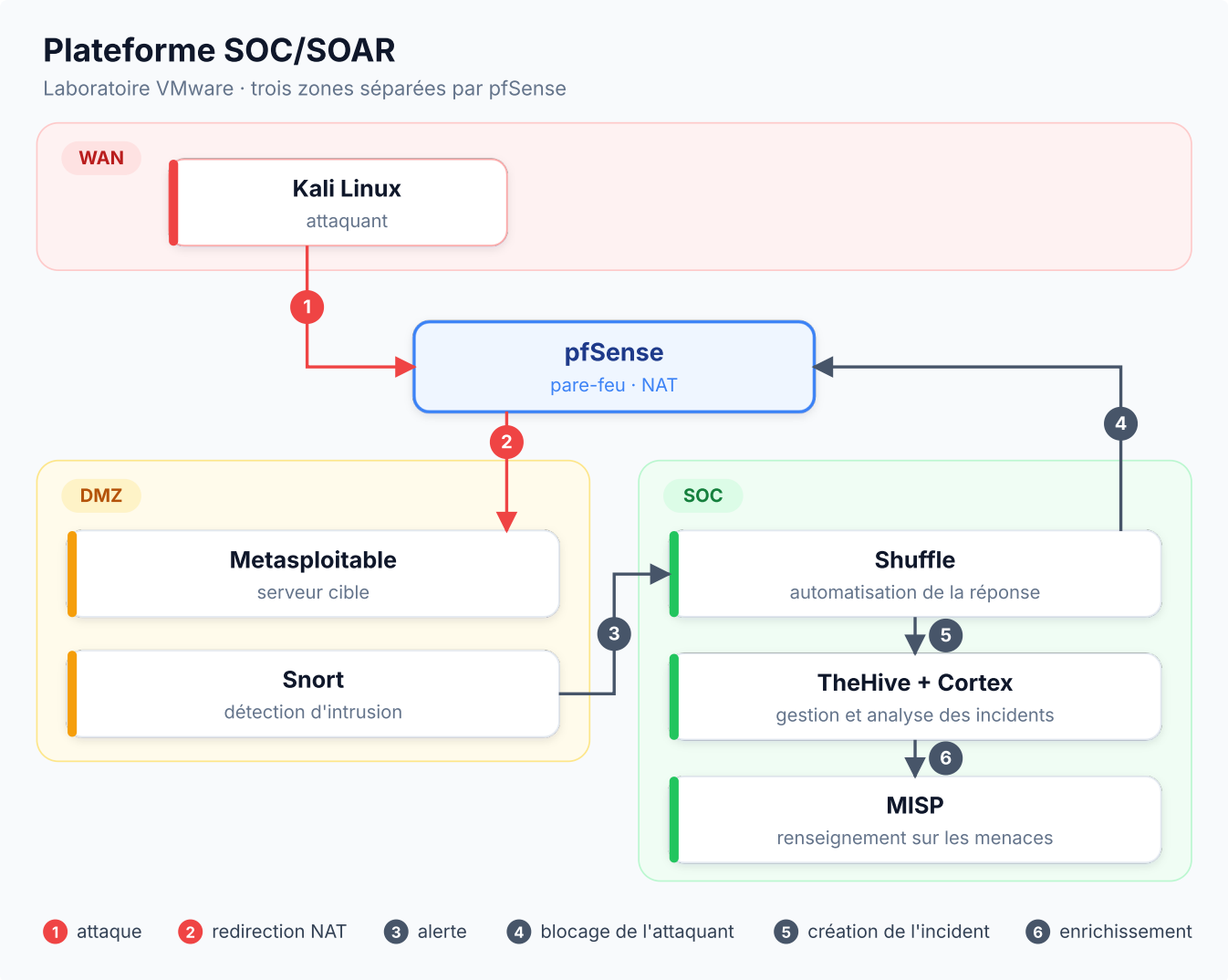


Figure 1 — Architecture de la plateforme SOC/SOAR

2.1 Réseaux

Réseau VMware	Type	Adressage	Rôle
VMnet0	pont (Wi-Fi)	DHCP de la box	zone WAN
LAN1	segment LAN	10.1.1.0/24	zone SOC
LAN2	segment LAN	10.1.2.0/24	zone DMZ
VMnet8	NAT	192.168.229.0/24	accès Internet des VM et administration depuis Windows

2.2 Machines virtuelles

Machine	Rôle	Adresse	RAM
pfSense	pare-feu, routeur, NAT	WAN (DHCP), LAN 10.1.1.1, DMZ 10.1.2.1	1 Go
Metasploitable	serveur cible vulnérable	10.1.2.100	0,5 Go
Snort	détection d'intrusion (IDS)	10.1.2.102	2 Go
Shuffle	SOAR, PostgreSQL, Grafana	10.1.1.150	6 Go
TheHive + Cortex	gestion et analyse des incidents	10.1.1.160	8 Go
MISP	threat intelligence	10.1.1.170	4 Go
Kali Linux	attaquant	WAN (DHCP)	2 Go

2.3 Pourquoi l'attaquant vise pfSense

Comme dans une entreprise réelle, le serveur de la DMZ n'a pas d'adresse publique : l'attaquant ne peut l'atteindre qu'à travers l'adresse WAN de pfSense. pfSense redirige ce trafic vers Metasploitable (redirection de ports, ou NAT). Tout le trafic d'attaque traverse donc la DMZ, où Snort l'observe. Pour les besoins des tests, tous les ports sont redirigés ; en production, seuls les services publiés (par exemple 80 et 443) le seraient.

3. Détection : pfSense, DMZ et Snort

3.1 pfSense

- Trois interfaces : **WAN** (réseau en pont), **LAN** (zone SOC, 10.1.1.1) et **DMZ** (10.1.2.1).
- *Block private networks* et *Block bogon networks* désactivés sur le WAN, car le WAN du laboratoire est lui-même un réseau privé.
- **Redirection de ports** du WAN vers Metasploitable (10.1.2.100).
- **Alias** `BLOCKED_IPS` (type *Host(s)*), alimenté automatiquement par Shuffle.
- **Règle de blocage** en tête des règles WAN : tout trafic dont la source appartient à `BLOCKED_IPS` est rejeté.
- **API REST v2** activée, avec authentification par clé (en-tête `x-api-key`), accessible uniquement depuis la zone SOC.

3.2 Snort

Snort est installé sur une VM Ubuntu de la DMZ (10.1.2.102). Il surveille le réseau `HOME_NET = 10.1.2.0/24` et écrit ses alertes dans `/var/log/snort/alert`. Quinze règles personnalisées couvrent les attaques testées :

Famille	Détection
ICMP	ping vers la DMZ, flood ICMP
Reconnaissance	scan de ports TCP SYN (Nmap)
Brute force	tentatives répétées sur FTP, SSH et Telnet
Web	flood HTTP, injection SQL (`OR 1=1`, `UNION SELECT`, sqlmap), XSS, injection de commande

3.3 Script de liaison Snort → Shuffle

Snort ne sait qu'écrire ses alertes dans un fichier. Un script Bash, exécuté comme service systemd (`snort-shuffle-watcher`), fait le lien avec Shuffle :

1. il lit en continu le fichier d'alertes de Snort ;
2. il ne conserve que les alertes des règles du projet ;
3. il ignore les adresses de l'infrastructure (liste blanche), pour ne jamais les bloquer ;
4. il évite les doublons : une même IP n'est signalée qu'une fois par minute ;
5. il envoie au webhook de Shuffle un message JSON, dont le champ `event_id` rend chaque alerte unique (TheHive refuse deux alertes de même référence) :

```
{ "attacker_ip": "192.168.100.60", "sid": "1001020",
  "alert": "TCP SYN port scan detected", "blocked_ips": ["192.168.100.60"],
  "event_id": "192.168.100.60-1001020-1792087160" }
```

4. Réponse automatisée avec Shuffle

4.1 Installation

Shuffle est déployé avec Docker sur une VM Ubuntu possédant deux interfaces : LAN1 (10.1.1.150, zone SOC) et NAT (accès Internet et administration). Ses composants sont le frontend (port 3001), le backend, Orborus (exécution des actions) et OpenSearch (stockage).

4.2 Le workflow de réponse



Figure 2 — Workflow de réponse automatique dans Shuffle

Étape	Action	Cible
Webhook	reçoit l'alerte envoyée par le script de Snort	Shuffle
Http 1	`PATCH /api/v2/firewall/alias` : ajoute l'IP de l'attaquant à `BLOCKED_IPS`	API pfSense
Http 2	`POST /api/v2/firewall/apply` : applique les règles	API pfSense
Grafana	`POST /api/ds/query` : enregistre l'incident en base	Grafana → PostgreSQL
Http 3	`POST /thehive/api/v1/alert` : crée l'alerte	TheHive
Gmail	envoie une notification à l'équipe	messagerie

Les variables `$exec.attacker_ip`, `$exec.sid`, `$exec.alert` et `$exec.event_id`, issues du message JSON, sont réutilisées dans chaque bloc.

4.3 Journalisation avec PostgreSQL et Grafana

PostgreSQL et Grafana sont installés sur la VM Shuffle. Chaque incident est inséré dans la table `incidents` (date, IP source, type d'attaque, sévérité, action réalisée). Un tableau de bord Grafana présente l'historique des attaques bloquées.

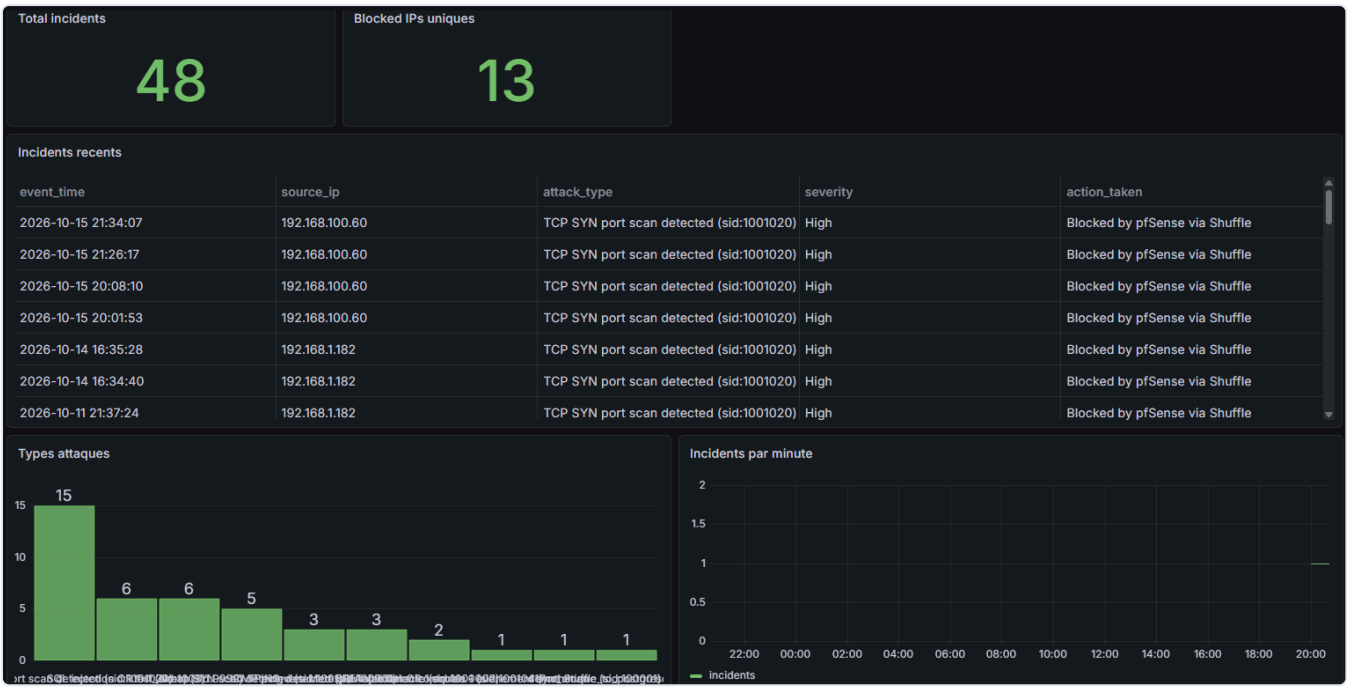


Figure 3 — Tableau de bord Grafana : nombre d'incidents, IP bloquées, incidents récents et types d'attaques

5. Gestion des incidents avec TheHive

5.1 Déploiement

TheHive 5.8 et Cortex 4.1 sont installés ensemble sur une VM Ubuntu Server 22.04 (8 Go de RAM, 4 processeurs) avec le kit Docker officiel de StrangeBee :

```
git clone https://github.com/StrangeBeeCorp/docker.git
cd docker/testing
bash ./scripts/init.sh
docker compose up -d
```

Cinq conteneurs sont lancés : **Cassandra** (base de données de TheHive), **Elasticsearch** (index de TheHive et base de Cortex), **TheHive** (port 9000), **Cortex** (port 9001) et **Nginx**, point d'entrée HTTPS.

5.2 Comptes et séparation des rôles

Profil	Type	Rôle
admin	normal	administrer la plateforme : organisations, utilisateurs, connecteurs
analyst	normal	traiter les alertes et les cas
org-admin	service	compte technique utilisé par Shuffle pour créer les alertes par l'API

Le profil *analyst* ne permet pas de créer des alertes par l'API. Plutôt que d'utiliser le compte d'un analyste, un **compte de service** dédié à Shuffle a été créé, avec sa propre clé API : c'est le principe du moindre privilège et de la séparation des rôles.

5.3 Intégration avec Shuffle

Le bloc Http 3 du workflow envoie à TheHive le corps suivant, authentifié par la clé du compte de service :

```
{ "type": "snort", "source": "Snort-IDS", "sourceRef": "$exec.event_id",
  "title": "$exec.alert", "severity": 3, "tags": ["snort", "shuffle"],
  "observables": [ { "dataType": "ip", "data": "$exec.attacker_ip" } ] }
```

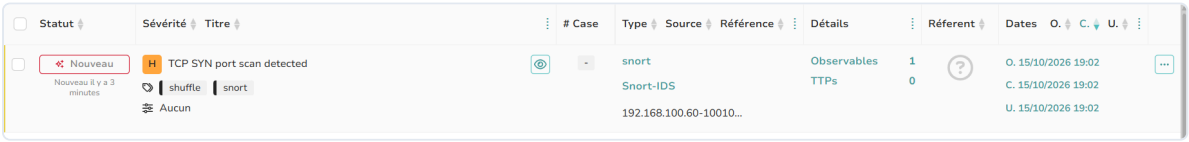


Figure 4 — Alerte créée automatiquement dans TheHive, avec l'IP de l'attaquant en observable

5.4 Travail de l'analyste

L'analyste examine l'alerte, la transforme en **cas**, lui assigne des tâches (vérifier le blocage, vérifier l'état du serveur cible) et marque l'IP de l'attaquant comme **IOC** une fois l'attaque confirmée. TheHive peut générer un rapport d'incident pour chaque cas.

6. Analyse des observables avec Cortex

6.1 Rôle de Cortex

Cortex est le moteur d'analyse de TheHive. Depuis un observable, l'analyste lance un ou plusieurs **analyseurs**. Chacun interroge une source d'information et renvoie un verdict : *info*, *safe*, *suspicious* ou *malicious*.

1. L'analyste choisit les analyseurs sur un observable dans TheHive.
2. TheHive transmet la demande à Cortex avec sa clé API.
3. Cortex vérifie les niveaux TLP et PAP, puis lance l'analyseur dans un conteneur Docker.
4. L'analyseur interroge sa source, produit son rapport, puis s'arrête.
5. TheHive affiche le résumé du rapport sur l'observable.

6.2 Analyseurs utilisés

Analyseur	Information fournie
MaxMind GeoIP	localisation géographique d'une adresse IP
DShield	réputation publique d'une IP (activité d'attaque signalée par la communauté SANS)
MISP	présence de l'IP dans la base de threat intelligence du SOC

> **Choix de conception.** Un premier test sur l'IP publique 8.8.8.8 a renvoyé un verdict *suspicious* de DShield, alors qu'il s'agit d'un serveur DNS légitime : l'IP figure dans des listes de surveillance générales. Un verdict de réputation seul ne doit donc pas déclencher de blocage automatique. Dans cette plateforme, le blocage repose uniquement sur une attaque **réellement observée** par Snort.

7. Threat Intelligence avec MISP

7.1 Déploiement

MISP 2.5 est déployé avec le projet officiel `misp-docker` sur une VM dédiée (4 Go de RAM, 10.1.1.170). Six conteneurs le composent : `misp-core`, `misp-modules`, la base MariaDB, Valkey (cache), un serveur mail et Nginx. Un certificat auto-signé, valable pour les deux adresses de la VM, sécurise l'accès HTTPS.

7.2 Les deux liaisons

Liaison	Sens	Fonctionnement
Cortex → MISP	lecture	l'analyseur MISP recherche l'observable dans MISP, avec une clé en **lecture seule**
TheHive → MISP	écriture	l'analyste exporte un cas vers MISP depuis TheHive, avec une clé autorisée en **écriture**

Deux clés distinctes sont utilisées : chaque outil ne dispose que des droits dont il a besoin.

7.3 Une base de renseignement validée par l'analyste

L'export vers MISP n'est pas automatique : seul l'analyste, après avoir confirmé l'attaque, marque l'observable comme **IOC** et exporte le cas. MISP ne contient ainsi que des indicateurs vérifiés, sans faux positifs. Une correspondance dans MISP devient alors un critère fiable.

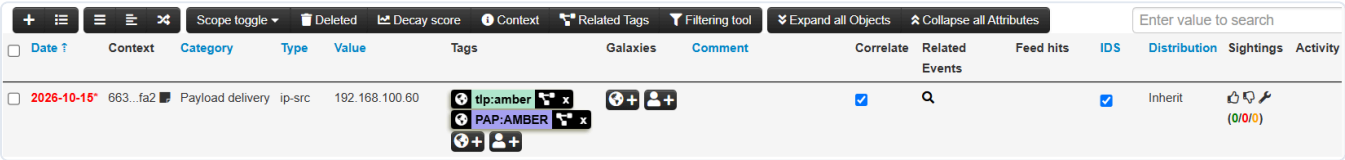


Figure 5 — L'IP de l'attaquant, exportée depuis TheHive, enregistrée dans MISP (type ip-src, marquée IDS)

TheHive transmet automatiquement le type d'attribut (`ip-src`), les niveaux de partage TLP et PAP, et active les options **IDS** (indicateur utilisable pour la détection) et **Correlate** (MISP relie les événements qui partagent un même indicateur).

8. Scénario de validation de bout en bout

Le scénario suivant a été rejoué sur la plateforme complète. L'attaquant (Kali, 192.168.100.60) scanne l'adresse WAN de pfSense (192.168.100.110), redirigée vers Metasploitable.

Étape 1 — Attaque et blocage

```
sudo nmap -sS -Pn -T4 192.168.100.110
```

Le premier scan aboutit en moins d'une seconde et découvre les services ouverts de Metasploitable (FTP, SSH, Telnet, HTTP, MySQL…). Snort détecte le scan, Shuffle reçoit l'alerte et bloque l'attaquant. Le scan suivant ne reçoit plus aucune réponse : les 1000 ports apparaissent *filtered*.



Figure 6 — Vu de l'attaquant : ports ouverts avant le blocage, puis 1000 ports filtrés après

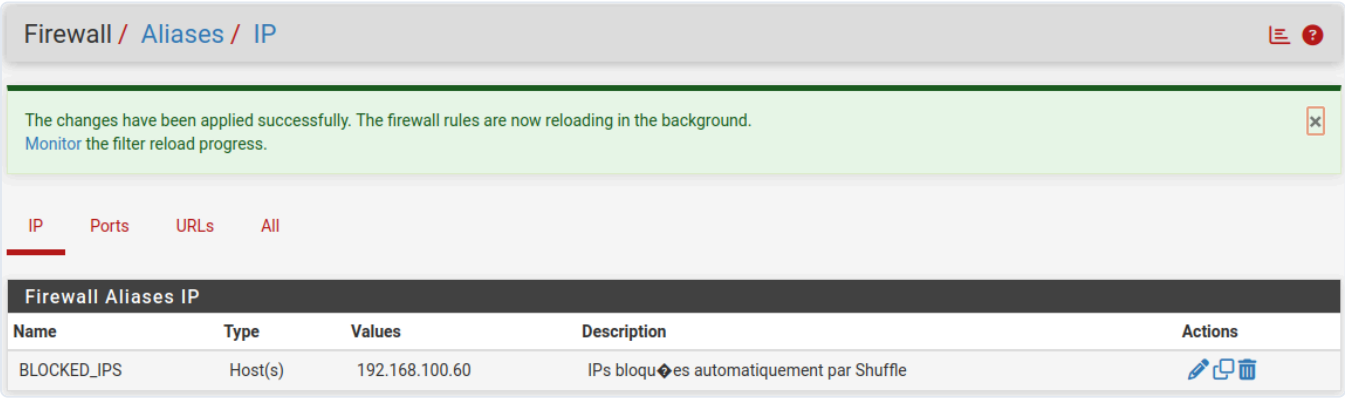


Figure 7 — L'IP de l'attaquant ajoutée automatiquement à l'alias BLOCKED_IPS de pfSense

Étape 2 — Gestion de l'incident

Simultanément, l'alerte « TCP SYN port scan detected » apparaît dans TheHive (figure 4). L'analyste la transforme en cas, confirme l'attaque, marque l'IP comme IOC et exporte le cas vers MISP (figure 5).

Étape 3 — Enrichissement et reconnaissance

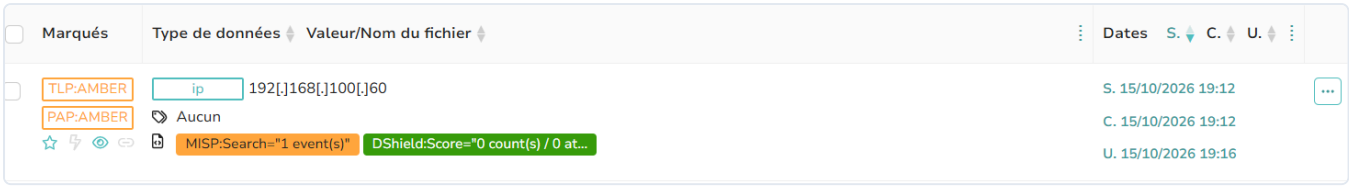


Figure 8 — Résultats des analyseurs Cortex sur l'IP de l'attaquant

L'analyseur **MISP** retrouve l'attaquant (*1 event*), alors que la source publique **DShield** ne le connaît pas (*0 count*). La base de renseignement interne reconnaît donc une menace que les sources publiques ignorent : c'est la valeur ajoutée de la threat intelligence propre au SOC.

Synthèse des tests

Scénario	Résultat
Scan de ports Nmap	détecté, attaquant bloqué en moins d'une minute
Brute force Telnet (Hydra)	détecté et bloqué
Injection SQL (Mutillidae)	détectée et bloquée
Gestion d'incident	alerte créée automatiquement dans TheHive, transformée en cas
Threat intelligence	IOC exporté vers MISP, attaquant reconnu par Cortex

9. Difficultés rencontrées et solutions

La plupart des difficultés provenaient de trois causes : les performances de la virtualisation, les droits de fichiers créés par Docker et des contrôles de démarrage trop stricts.

Composant	Problème	Cause	Solution
VMware	VM très lentes, blocages	Hyper-V et sécurité basée sur la virtualisation actifs	désactivation d'Hyper-V et de l'intégrité de la mémoire
VMware	VM refusée au démarrage	mémoire réservée dépassée	RAM des VM ajustée
Snort	interface inactive après redémarrage	`systemd-networkd` non activé	activation du service
Shuffle	exécutions bloquées	disque saturé par OpenSearch	disque agrandi de 20 à 80 Go
TheHive	Cassandra et Elasticsearch *unhealthy*	contrôle de santé limité à 1 s	délais portés à 10 s, 10 tentatives
TheHive	indisponible après redémarrage	démarre avant Elasticsearch	redémarrage automatique différé (tâche `@reboot`)
Cortex	accès refusé à Docker	socket Docker attribué à l'utilisateur 1001	groupe dédié (gid 1001)
Cortex	analyseurs en échec	Docker 29 refuse l'ancienne API ; dossier de travail en root	`DOCKER_MIN_API_VERSION=1.24` ; droits corrigés
Shuffle → TheHive	alertes en double refusées	référence identique	identifiant unique `event_id`
MISP	Nginx redémarre en boucle	HTTPS sans certificat	certificat auto-signé
TheHive → MISP	événement exporté vide	observable non marqué IOC	marquage IOC avant l'export
Laboratoire	attaque sans effet après changement de réseau Wi-Fi	nouvelles adresses DHCP du WAN	attaque relancée sur la nouvelle adresse de pfSense

10. Conclusion et perspectives

10.1 Bilan

Ce projet a permis de construire une plateforme de sécurité complète à partir d'outils open source, déployée et validée dans un laboratoire virtualisé. Elle couvre l'ensemble du cycle de traitement d'un incident :

- **Détection** : Snort identifie les scans, les attaques par force brute et les attaques web ;
- **Réponse automatisée** : Shuffle bloque l'attaquant sur pfSense en moins d'une minute, sans intervention humaine ;
- **Gestion des incidents** : TheHive centralise les alertes et organise le travail des analystes ;
- **Threat intelligence** : Cortex enrichit les observables et MISP capitalise les indicateurs confirmés.

Deux principes ont guidé la conception : n'automatiser le blocage que sur une preuve forte (une attaque réellement observée), et ne faire entrer dans la base de renseignement que des indicateurs validés par un analyste.

10.2 Compétences acquises

Configuration d'un pare-feu et segmentation réseau, écriture de règles IDS, automatisation par API REST, déploiement de services avec Docker, intégration d'outils SOC entre eux, et résolution de problèmes système en environnement Linux.

10.3 Perspectives

- Interroger automatiquement Cortex et MISP depuis Shuffle avant chaque décision, pour adapter la réponse au niveau de menace ;
- activer des flux publics de threat intelligence dans MISP (par exemple CIRCL OSINT) ;
- étendre les playbooks à d'autres types d'attaques et d'actions de réponse ;
- remplacer les certificats auto-signés par une autorité de certification interne.